

home_user.js — Fundamentos

Explicación detallada del inicio del alumno (con el nuevo estilo de secciones)

Es la pantalla de **inicio** del alumno (primera ruta del grupo “usuario común” en app/index.tsx) — un dashboard simple con saludo, una estadística, y dos accesos rápidos a acciones del día.

1. Guard clause + hooks

```
export default function UserHomeScreen({ navigation }) {  
  const { isLandscape } = useResponsiveLayout();  
  const { user } = useAuth();
```

```
  if (!user) return null;
```

Mismo patrón ya visto en mi_cuenta_trainer.js/mi_cuenta_user.js: lee user del contexto de auth y corta con return null si por algún motivo se renderizara sin sesión activa (transición de logout). Acá no se desestructura logout porque esta pantalla no tiene botón de cerrar sesión — eso vive en mi_cuenta_user.js.

2. Saludo

```
<Text style={styles.greeting}> Hola, {user.firstName}! </Text>  
<Text style={styles.subtitle}> Listo para destruir tus metas de hoy? </Text>
```

Interpolación directa de user.firstName en el saludo — dato que viene de users.js (recordá que addUser lo guarda en el registro, separado de lastName).

3. El patrón sectionBox — las tres secciones ahora unificadas

Esta es la parte más importante de la versión actual. Las tres secciones (Estadísticas, Progreso Diario, Tu próximo entrenamiento) comparten exactamente la misma estructura JSX:

```
<View style={styles.sectionBox}>  
  <Text style={styles.sectionTitle}> Título de la sección </Text>  
  <View style={styles.filaInterna}>  
    /* contenido específico de la sección */  
  </View>  
</View>
```

Y el estilo compartido:

```
sectionBox: {  
  backgroundColor: '#1D1D1D',  
  borderRadius: 16,  
  borderWidth: 1.5,  
  borderColor: '#FFE082',  
  padding: '5.5%',  
  marginBottom: '5%',  
}
```

Cambio de diseño aplicado: antes, cada sección tenía su propia mezcla de “título flotante afuera + tarjeta con su propio fondo/radio adentro”, y “Estadísticas” ni siquiera tenía título visible. Ahora **título y contenido son una sola unidad visual**, con un contenedor consistente que se repite tres veces con distinto contenido interno.

Es el mismo principio de “un solo estilo compartido, contenido variable” que ya se ve en ProfileDataCard (una tarjeta reutilizable), solo que acá no se extrajo a un componente aparte porque

cada sección tiene contenido interno bastante distinto entre sí (una fila de stats, una fila de progreso con botón, una fila de rutina con botón) — no serían filas homogéneas como en ProfileDataCard, así que se mantuvo como estilo compartido (sectionBox) en vez de componente.

El borderColor: '#FFE082' (amarillo claro/pálido) es deliberadamente **distinto** al #FFC107 (amarillo fuerte) que usan los botones (smallButton, trainButton) — así el borde decora sin competir visualmente con los elementos interactivos de la pantalla.

4. Sección “Estadísticas”

```
<View style={styles.sectionBox}>
  <Text style={styles.sectionTitle}> Estadísticas </Text>
  <View style={styles.statsRow}>
    <View style={styles.statCard}>
      <Text style={styles.statIcon}> ■ </Text>
      <Text style={styles.statNumber}> 0 </Text>
      <Text style={styles.statLabel}> Días en racha </Text>
    </View>
  </View>
</View>
```

statsRow usa flexDirection: 'row' aunque hoy solo tenga **una** tarjeta (statCard con flex: 1) — esto es intencional pensando en el futuro: la lógica de “días en racha” todavía no está implementada (el número está **hardcodeado en 0**), y cuando se implemente (sumar un día cuando el alumno escanea el QR del gimnasio al entrar, si son días consecutivos), es muy probable que se agreguen más estadísticas a esta fila (por ejemplo “entrenamientos totales”, “mejor racha”), y con flexDirection: 'row' + flex: 1 en cada tarjeta, alcanza con agregar más <View style={styles.statCard}> para que se repartan el espacio automáticamente sin tocar el layout.

5. Sección “Progreso Diario”

```
<View style={styles.sectionBox}>
  <Text style={styles.sectionTitle}> Progreso Diario </Text>
  <View style={styles.progressRow}>
    <View style={styles.progressInfo}>
      <Text style={styles.progressTitle}> ¿Ya entrenaste hoy? </Text>
      <Text style={styles.progressSubtitle}> Escanea el QR para no perder la racha. </Text>
    </View>
    <TouchableOpacity style={styles.smallButton}>
      <Text style={styles.smallButtonText}> Escanear </Text>
    </TouchableOpacity>
  </View>
</View>
```

Punto importante para la defensa: el botón “Escanear” **no tiene onPress**. Es una pieza de UI ya diseñada pero todavía sin funcionalidad — coherente con que la lógica de escanear el QR y sumar el día de racha es una feature a implementar más adelante. Hoy es un placeholder visual que anticipa esa función futura, no un botón roto por error.

progressInfo tiene flex: 1 mientras que el botón no, así el texto ocupa todo el espacio disponible y el botón queda con su tamaño natural pegado a la derecha (patrón flexbox típico de “contenido flexible + acción de tamaño fijo al costado”).

6. Sección “Tu próximo entrenamiento”

```
<View style={styles.sectionBox}>
  <Text style={styles.sectionTitle}> Tu próximo entrenamiento </Text>
```

```

<View style={styles.workoutRow}>
  <View style={styles.workoutInfo}>
    <Text style={styles.workoutTitle}> Tu rutina </Text>
    <Text style={styles.workoutSubtitle}> Clickea para ver los ejercicios de tu rutina. </Text>
  </View>
  <TouchableOpacity style={styles.trainButton} onPress={() => navigation.navigate('mi_rutina_user')}>
    <Text style={styles.trainIcon}> ■ </Text>
    <Text style={styles.trainButtonText}> A entrenar </Text>
  </TouchableOpacity>
</View>
</View>

```

Esta sí tiene navegación funcional: `navigation.navigate('mi_rutina_user')` lleva a la pantalla de rutina del alumno (la que ya está conectada con `routines.js/exercises.js`). Misma estructura que “Progreso Diario” (Info con flex: 1 + botón fijo), pero acá el título del botón dice “Tu rutina” en genérico, sin mostrar el nombre real de la rutina (`routine.name`) — otra oportunidad de mejora futura, ya que la pantalla no llama a `getRoutineById(user.id_rutina)` en ningún momento.

7. El bug corregido: estilo duplicado

Corrección aplicada: en el StyleSheet original había **`nextWorkoutCard` definido dos veces**. En un objeto literal de JavaScript, cuando una clave se repite, **la segunda definición pisa silenciosamente a la primera**, sin ningún error ni warning del compilador — la primera definición (con menos padding, sin `minHeight`) nunca se aplicaba, solo la segunda tenía efecto real. Ahora se unificó todo en un único `workoutRow`, así el estilo queda inequívoco y no hay ambigüedad para quien lea el código de qué valores realmente se están aplicando.

Resumen para el oral

“home_user.js es el dashboard del alumno: saludo personalizado más tres secciones (Estadísticas, Progreso Diario, Próximo entrenamiento), todas construidas con el mismo patrón visual `sectionBox` — un contenedor con borde amarillo claro que agrupa título y contenido, en vez de tener el título flotando afuera de la tarjeta como antes. El número de 'días en racha' está `hardcodeado` en 0 y el botón 'Escanear' no tiene `onPress` todavía, porque la lógica de sumar racha al escanear un QR al entrar al gimnasio es una funcionalidad pendiente. Solo el botón 'A entrenar' está conectado, navegando a `mi_rutina_user`. De paso, se corrigió un bug real: el StyleSheet tenía la clave `nextWorkoutCard` duplicada, donde la segunda definición pisaba silenciosamente a la primera.”